



POO - Unid II

Exercício 1:

Considere o código abaixo:

```
using System;
```

```
// Classe abstrata base
```

```
public abstract class Forma
```

```
{
```

```
    public string Nome { get; protected set; }
```

```
    // Construtor da classe base abstrata
```

```
    protected Forma(string nome) {
```

```
        Nome = nome;
```

```
    }
```

```
    // Membro abstrato: nenhuma implementação aqui
```

```
    public abstract double CalcularArea();
```

```
    // Método concreto: implementado na classe abstrata
```

```
    public void Descrever()
```

```
    {
```

```
        Console.WriteLine($"Esta forma é um {Nome}.");
```

```
    }
```

```
}
```

```
// Subclasse concreta da forma
```

```
public class Circulo : Forma
```

```
{
```

```
    private double raio;
```

```
    public Circulo(double raio) : base("círculo") {
```

```
        this.raio = raio;
```

```
    }
```

```
    // Implementação obrigatória do método abstrato
```

```
    public override double CalcularArea()
```

```
    {
```

```
        return Math.PI * raio * raio;
```

```
    }
```

```
}
```

```
// Outra subclasse concreta
```

```
public class Retangulo : Forma
```

```
{
```

```
    private double largura, altura;
```

```
public Retangulo(double largura, double altura) : base("retângulo") {  
    this.largura = largura;  
    this.altura = altura;  
}  
  
public override double CalcularArea()  
{  
    return largura * altura;  
}  
}
```

Você precisa somar áreas em um algoritmo que deve aceitar (i) objetos que reaproveitam o estado comum e o método Descrever() e (ii) objetos que não podem herdar de Forma (porque já herdam de outra classe), mas, ainda assim, devem participar do mesmo polimorfismo. Qual mudança atende a esse cenário, mantendo o contrato de "toda figura sabe calcular área" verificado em compilação e sem exigir duplicação do código compartilhado de Forma?

A)

Transformar Forma em classe concreta e implementar CalcularArea() retornando 0.0, para que classes externas possam herdar e "corrigir" depois.

B)

Introduzir uma interface IForma com double CalcularArea();, fazer Forma : IForma e manter CalcularArea() como abstrato em Forma, passando o algoritmo a operar sobre IForma.

C)

Manter apenas a classe abstrata Forma e exigir que qualquer tipo participante herde de Forma, mesmo que isso imponha múltipla herança de classes.

D)

Mover Nome e Descrever() para uma interface e adicionar campos de instância na interface para armazenar o nome compartilhado.

E)

Remover CalcularArea() do modelo e calcular áreas por reflexão a partir de nomes de tipos, preservando o encapsulamento por não depender de membros públicos.

Exercício 2:

Um cliente recebe um objeto por um tipo base e assume que pode ajustar duas dimensões de forma independente antes de calcular área. Em uma tentativa de representar um quadrado como subtipo de retângulo, o tipo derivado força as duas dimensões a permanecerem iguais, alterando o efeito de atribuições separadas. Qual opção corrige o problema de substituição sem impor ao cliente do tipo base novas restrições silenciosas?

A)

Manter a herança e, no tipo derivado, lançar exceção sempre que o cliente tentar setar largura e altura com valores diferentes.

B)

Manter a herança e, no tipo derivado, aceitar os setters, mas "consertar" silenciosamente um dos valores para igualar o outro.

C)

Manter a herança e tornar os setters do tipo base protected, forçando que só subclasses consigam ajustar dimensões livremente.

D)

Remover a relação de herança entre quadrado e retângulo e fazer ambos aderirem a um contrato comum (por exemplo, uma interface de cálculo de área), reservando a herança de classes para reuso que não altere expectativas do cliente.

E)

Manter a herança e marcar o tipo derivado como sealed, para impedir novas variações e, assim, evitar efeitos inesperados.

Exercício 3:

Considere exatamente o código C# a seguir, distribuído em dois arquivos na mesma solução:

Arquivo CumprimentoMulti.cs:

```
interface IGreetingsEn {
    string Saudar();
}
interface IGreetingsFr {
    string Saudar();
}

class CumprimentoMulti : IGreetingsEn, IGreetingsFr {
    // Implementação implícita para inglês:
    public string Saudar() {
        return "Hello"; // Saudação em inglês por padrão
    }
    // Implementação explícita para francês:
    string IGreetingsFr.Saudar() {
        return "Bonjour"; // Saudação em francês somente via interface
    }
}
```

Arquivo Program.cs (com instruções de nível superior):

```
using System;

var obj = new CumprimentoMulti();
Console.WriteLine(obj.Saudar()); // Saída: Hello
Console.WriteLine(((IGreetingsEn)obj).Saudar()); // Saída: Hello (IGreetingsEn
usa implícita)
Console.WriteLine(((IGreetingsFr)obj).Saudar()); // Saída: Bonjour (IGreetingsFr
usa explícita)
```

Qual alternativa descreve corretamente, ao mesmo tempo, (i) a vinculação em tempo de compilação do método chamado em `obj.Saudar()` e (ii) a razão pela qual a saudação em francês exige conversão explícita para a interface correspondente?

A)

A chamada `obj.Saudar()` é vinculada ao membro explícito de `IGreetingsFr`, porque membros de interfaces têm precedência sobre membros públicos da classe; a conversão só é exigida para evitar ambiguidade em tempo de execução.

B)

A chamada `obj.Saudar()` é vinculada ao método público da classe, porque o membro explícito não integra a superfície pública do tipo concreto; a conversão é exigida porque o membro explícito só é acessível quando a referência está tipada como `IGreetingsFr`.

C)

A chamada `obj.Saudar()` é vinculada dinamicamente em tempo de execução, escolhendo a versão "mais específica" entre as interfaces; a conversão é exigida porque `Console.WriteLine` não aceita chamadas a membros de interfaces sem cast.

D)

A chamada `obj.Saudar()` falha em compilação devido à colisão de assinaturas entre as interfaces; a conversão funciona apenas por causa de boxing implícito do objeto para as interfaces.

E)

A chamada `obj.Saudar()` invoca sempre a implementação explícita, mas o resultado "Hello" ocorre por uma regra de fallback do runtime; a conversão é exigida porque métodos explícitos não podem retornar string diretamente.

Exercício 4:

Suponha uma interface `IRepositorio` cujo contrato, na documentação, promete: "ao chamar `Remover(item)`, o item deve ser removido se ele existir no repositório". Sem alterar a assinatura do método nem o tipo de retorno, qual alternativa descreve uma implementação que viola a substituíbilidade exigida pelo LSP (Princípio da Substituição de Liskov), mesmo que o código compile?

A)

A implementação remove o item quando ele existe e, quando ele não existe, apenas não faz nada (isto é, torna a operação idempotente).

B)

A implementação lança uma exceção `NotSupportedException` para qualquer chamada, inclusive quando o item existe; dessa forma, a remoção prometida pelo contrato deixa de acontecer.

C)

A implementação remove o item quando ele existe e registra a operação em log; quando ele não existe, registra um aviso e não altera o repositório.

D)

A implementação remove o item quando ele existe e, quando ele não existe, também não altera o repositório, mas atualiza uma métrica interna de tentativas.

E)

A implementação remove o item quando ele existe e, quando ele não existe, retorna silenciosamente; adicionalmente, garante que nenhuma exceção nova é introduzida em relação ao contrato documentado.

Exercício 5:

Considere o código C# a seguir, distribuído em dois arquivos na mesma solução:

Arquivo Retangulo.cs:

```
class Retangulo {  
    public virtual void DefinirLargura(double larg) { Largura = larg; }  
    public virtual void DefinirAltura(double alt) { Altura = alt; }  
    public double Largura { get; protected set; }  
    public double Altura { get; protected set; }  
    public virtual double Area => Largura * Altura;  
}
```

```
class Quadrado : Retangulo {  
    public override void DefinirLargura(double larg) {  
        base.DefinirLargura(larg);  
        base.DefinirAltura(larg);  
    }  
    public override void DefinirAltura(double alt) {  
        base.DefinirAltura(alt);  
        base.DefinirLargura(alt);  
    }  
}
```

Arquivo Program.cs (com instruções de nível superior):

```
using System;  
  
void AumentarRetanguloEmDobro(Retangulo r) {  
    r.DefinirLargura(r.Largura * 2);  
    r.DefinirAltura(r.Altura * 2);  
    // Esperamos que a área agora seja 4 vezes maior:  
    Console.WriteLine("Área quadruplicada: " + r.Area);  
}  
  
// Cenário de uso:  
Retangulo meuRet = new Retangulo();
```

```
meuRet.DefinirLargura(5);  
meuRet.DefinirAltura(10);  
// Área inicial = 50  
AumentarRetanguloEmDobro(meuRet);  
// Área esperada = 200 (50 * 4) -> correto para Retangulo  
  
Retangulo meuQuad = new Quadrado();  
meuQuad.DefinirLargura(5);  
meuQuad.DefinirAltura(10);  
// Aqui, ao definir altura para 10, a largura também se torna 10 (Quadrado impõe  
lados iguais)  
// Área inicial de meuQuad = 100 (10*10)  
AumentarRetanguloEmDobro(meuQuad);
```

Mantendo a lógica do código como está, qual é o valor numérico efetivamente impresso na segunda chamada de `AumentarRetanguloEmDobro(meuQuad)` (isto é, o valor de `r.Area` naquele `Console.WriteLine`)?

A)

Ele é 200, pois a função dobra duas dimensões e, portanto, multiplica a área por quatro, independentemente do subtipo.

B)

Ele é 400, pois a referência é do tipo base e apenas a largura é ajustada pelo subtipo na primeira chamada.

C)

Ele é 800, pois o ajuste do quadrado duplica o efeito de apenas uma das atribuições e, então, a área cresce oito vezes.

D)

Ele é 1600, pois as duas chamadas (dobrar a largura e depois dobrar a altura) acabam dobrando duas vezes o lado do quadrado, levando-o a 40×40.

E)

O código não compila, porque `Quadrado` não pode sobrescrever dois métodos do tipo base preservando o tipo de retorno de `Area`.

Exercício 6:

Em um jogo, existe uma classe abstrata Personagem com métodos concretos que mantêm invariantes (por exemplo, "a vida nunca pode ficar negativa" e "o estado de morte só é acionado uma vez"). O método público ReceberDano(int) é virtual, e várias subclasses o sobrescrevem, algumas chamando base.ReceberDano(dano) e outras não. Após uma alteração pequena na superclasse (para registrar telemetria e ajustar a ordem de atualização do estado), surgem regressões em subclasses que antes funcionavam. Qual mudança de projeto tende a reduzir de forma mais direta esse tipo de regressão em cascata, sem eliminar a reutilização de código comum?

A)

A decisão de manter ReceberDano(int) como virtual, mas de tornar os campos internos da classe base private e de expor o estado apenas por propriedades públicas.

B)

A decisão de transformar ReceberDano(int) em um método não virtual que aplica o algoritmo completo e de delegar a variação a um ou mais ganchos protected (abstract ou virtual) para etapas específicas, evitando que subclasses alterem o fluxo principal.

C)

A decisão de introduzir uma nova subclasse intermediária abstrata (por exemplo, PersonagemComDefesa) para concentrar as mudanças, aumentando a profundidade da hierarquia para acomodar variações futuras.

D)

A decisão de manter o modelo atual, mas de marcar todas as subclasses concretas como sealed, para impedir novas derivações e, assim, diminuir o risco de efeitos colaterais.

E)

A decisão de substituir abstract class Personagem por uma interface IPersonagem e de mover toda a lógica comum para métodos de extensão estáticos, eliminando o uso de herança de classes.

Exercício 7:

Considere o código C# a seguir:

```
using System;

abstract class Personagem {
    public virtual void Falar() {
        Console.WriteLine("Base");
    }
}

class Guerreiro : Personagem {
    public sealed override void Falar() {
        Console.WriteLine("Guerreiro");
    }
}

class Paladino : Guerreiro {
    public new void Falar() {
        Console.WriteLine("Paladino");
    }
}

class Program {
    static void Main() {
        Personagem p = new Paladino();
        p.Falar();

        Paladino q = new Paladino();
        q.Falar();
    }
}
```

Qual alternativa descreve corretamente o comportamento do programa e o motivo técnico relevante?

A)

O código não compila, porque sealed override proíbe qualquer membro com o mesmo nome em classes derivadas, mesmo com new.

B)

O código compila, imprime "Guerreiro" e depois "Paladino", porque a chamada via Personagem usa despacho virtual até o último override permitido (selado em Guerreiro), enquanto a chamada via Paladino resolve para o método ocultador definido com new.

C)

O código compila, imprime "Paladino" duas vezes, porque, em tempo de execução, o CLR sempre escolhe o método mais derivado, independentemente de sealed.

D)

O código compila, imprime "Base" e depois "Paladino", porque sealed override cancela o polimorfismo e restaura a implementação do tipo base original.

E)

O código compila, imprime "Guerreiro" duas vezes, porque new não altera a resolução de chamadas, nem em tempo de compilação nem em tempo de execução.

Exercício 8:

Em um jogo, personagens podem combinar capacidades ortogonais (por exemplo, "voar", "atacar a distância", "ser invisível", "aplicar veneno"), e essas capacidades podem ser ganhas ou perdidas durante a partida. Há também a restrição de que uma classe concreta só pode herdar de uma única superclasse concreta. Qual proposta de modelagem tende a evitar a explosão combinatória de subclasses e a permitir recombinações em tempo de execução com menor acoplamento estrutural?

A)

A decisão de criar uma hierarquia profunda em que cada nível adiciona exatamente uma capacidade, de modo que toda combinação seja representada por uma subclasse final específica.

B)

A decisão de manter a hierarquia rasa, mas de duplicar o código das capacidades em cada classe concreta que precisar delas, para reduzir dependências de superclasses.

C)

A decisão de representar cada capacidade como um componente (objeto) com uma interface estável, e de fazer a entidade do jogo manter um conjunto desses componentes, delegando a eles o comportamento conforme forem anexados ou removidos.

D)

A decisão de representar capacidades como métodos static em utilitários globais, e de passar a entidade como parâmetro, para eliminar a necessidade de objetos auxiliares.

E)

A decisão de colocar todas as capacidades na superclasse, controlando o uso por meio de flags e de condicionais, de forma que nenhuma subclasse precise existir.

Exercício 9:

Considere o código a seguir:

```
using System;
```

```
class Character {
    public string Name { get; set; }
    public Character(string name) {
        Name = name;
    }
    public virtual void Attack() {
        Console.WriteLine($"{Name} ataca de forma genérica.");
    }
}

class Warrior : Character {
    public Warrior(string name) : base(name) { }
    public override void Attack() {
        Console.WriteLine($"{Name} avança com sua espada, desferindo um golpe poderoso!");
    }
}

class Mage : Character {
    public Mage(string name) : base(name) { }
    public override void Attack() {
        Console.WriteLine($"{Name} lança uma bola de fogo ardente contra o inimigo!");
    }
}

// Código de demonstração
class Program {
    static void Main() {
        Character[] party = new Character[2];
        party[0] = new Warrior("Arus");
        party[1] = new Mage("Luna");
        foreach (Character member in party) {
```

```
        member.Attack();  
    }  
}
```

Agora, suponha que a palavra-chave virtual seja removida do método Attack() na classe Character, mantendo-se o restante do código exatamente como está. Qual alternativa descreve corretamente o que acontece?

A)

O código compila e o runtime decide, em tempo de execução, qual Attack() chamar; a saída permanece com as duas mensagens específicas das subclasses.

B)

O código compila, mas, como o método deixou de ser virtual, o loop chama sempre Character.Attack(), imprimindo duas vezes a mensagem genérica.

C)

O código não compila, porque as declarações override em Warrior e Mage deixam de corresponder a um membro virtual (ou abstrato) na classe base.

D)

O código compila, mas o runtime passa a usar despacho dinâmico apenas para interfaces; para classes, a escolha do método ocorre sempre em tempo de compilação.

E)

O código não compila apenas se Character.Attack() também for marcado como sealed; sem sealed, a sobrescrita continua válida.

Exercício 10:

Em um sistema, existe uma coleção do tipo IDamageable[] que contém objetos de classes diferentes, e o código executa target.TakeDamage(30) dentro de um foreach. Qual alternativa expressa corretamente a divisão entre o que é verificado em tempo de compilação e o que é decidido em tempo de execução, além do motivo de o acoplamento do código chamador ser considerado baixo?

A)

Em compilação, confirma-se apenas que a expressão target tem um método TakeDamage(int); em execução, a chamada é sempre resolvida para a implementação de IDamageable, independentemente do tipo real do objeto, pois interfaces não usam despacho polimórfico.

B)

Em compilação, confirma-se que o tipo estático (IDamageable) declara TakeDamage(int); em execução, o runtime seleciona a implementação concreta conforme o tipo real do objeto; o baixo acoplamento decorre de o chamador depender do contrato, não da classe concreta.

C)

Em compilação, o compilador fixa a chamada para o método da classe concreta armazenada no array; em execução, o runtime apenas executa esse endereço já fixado, e virtual/override seria irrelevante para interfaces.

D)

Em compilação, a chamada é aceita porque o compilador converte implicitamente target para dynamic; em execução, o binder escolhe qual método chamar com base no tipo real do objeto.

E)

Em compilação, o acoplamento é baixo porque o chamador enxerga todos os membros públicos do tipo concreto ao iterar o array; em execução, a interface serve só como "marca" (tag) e não participa do despacho.